

CC3092 - Deep Learning y Sistemas Inteligentes

Laboratorio #3

Redes Neuronales Recurrentes y LSTM

Instrucciones generales

- En parejas o individual.
- Entrega: domingo 23 de agosto, 2026. 23:59.

1. Dataset

Trabjarán con el dataset público IMDB Reviews, un problema de clasificación binaria de sentimiento (positivo / negativo) sobre reseñas de películas en texto. A diferencia de los laboratorios anteriores, aquí cada observación es una secuencia de longitud variable.

Dataset con HuggingFace datasets:

```
from datasets import load_dataset imdb = load_dataset("imdb")
```

2. Exploración y preparación de los datos

Cargue el dataset y responda las siguientes preguntas:

- ¿Cuántas observaciones y cuántas clases tiene el dataset? ¿Las clases están balanceadas?
- ¿Cuál es la longitud de las reseñas? Reporte mínimo, máximo, promedio y grafique la distribución.
- ¿Qué estrategia de tokenización usarán? ¿Qué tamaño de vocabulario obtienen y cómo manejarán las palabras fuera de vocabulario?
- ¿Por qué es necesario aplicar padding y/o truncamiento a las secuencias antes de agruparlas en batches? ¿Qué longitud máxima de secuencia eligieron y por qué?
- Visualice al menos 5 ejemplos del dataset (texto y etiqueta) junto con su longitud en tokens.

Divida el conjunto de datos en entrenamiento, validación y test.

3. Investigación: capas de PyTorch para RNN y LSTM

Investigue las capas y utilidades de torch.nn necesarias para construir y entrenar redes recurrentes. Para cada una, describa brevemente su propósito y sus parámetros más relevantes.

- nn.Embedding
- nn.RNN
- nn.LSTM
- nn.utils.rnn.pad_sequence, pack_padded_sequence y pad_packed_sequence
- torch.nn.utils.clip_grad_norm_ (gradient clipping)
- nn.Dropout aplicado a capas recurrentes

Adicionalmente, investigue brevemente los siguientes conceptos: diferencia entre estado oculto (hidden state) y estado de celda (cell state) en una LSTM; en qué consiste el problema de vanishing / exploding gradient y por qué ocurre con más severidad en secuencias largas; y cómo las compuertas (forget, input, output) de la LSTM ayudan a mitigar el vanishing gradient en comparación con una RNN simple.

4. Construcción y entrenamiento de las arquitecturas

Construya y entrene tres arquitecturas distintas para clasificar el sentimiento de las reseñas:

- Un MLP (baseline), que reciba una representación de tamaño fijo de la reseña, por ejemplo, el promedio (o suma) de los embeddings de sus palabras, o una representación bag-of-words.
- Una RNN simple (nn.RNN), que reciba la secuencia de embeddings palabra por palabra y utilice el estado oculto del último paso (arquitectura many-to-one) para la clasificación.
- Una LSTM (nn.LSTM), con la misma configuración many-to-one que la RNN, para comparar directamente el efecto de las compuertas de memoria.

Itere sobre cada arquitectura para obtener la mejor red posible.

Para cada iteración, registre:

- La configuración de hiperparámetros usada.
- La pérdida (loss) de entrenamiento y de validación por epoch.
- Las métricas de evaluación del problema de clasificación: accuracy, precision, recall y F1-score (macro o weighted), calculadas sobre el conjunto de validación.
- El número total de parámetros entrenables del modelo.

Grafiquen las curvas de pérdida de entrenamiento y validación de al menos 3 iteraciones por arquitectura (9 en total), para poder identificar visualmente señales de overfitting o underfitting.

Una vez identificada la mejor configuración de cada arquitectura según sus métricas de validación, evalúe los tres modelos finales una única vez sobre el conjunto de test, reporte sus métricas y genere la matriz de confusión de cada uno.

4.1 Experimento adicional: efecto de la longitud de secuencia

Usando las mejores configuraciones de la RNN y la LSTM, entréñenlas y evalúenlas sobre al menos dos versiones del dataset con longitudes máximas de secuencia distintas (por ejemplo, reseñas truncadas a 50 tokens vs. reseñas truncadas a 400-500 tokens). Comparen el desempeño de ambas arquitecturas en cada caso.

5. Comparación de arquitecturas

Con los resultados de la mejor configuración de cada arquitectura, realicen una comparación directa entre el MLP, la RNN y la LSTM, considerando:

- Cantidad de parámetros entrenables de cada modelo.
- Desempeño (accuracy, precision, recall, F1-score) sobre el conjunto de test.
- Relación entre la cantidad de parámetros y la calidad de los resultados obtenidos.
- Tiempo de entrenamiento aproximado de cada arquitectura.
- Resultados del experimento de longitud de secuencia (sección 4.1) para RNN y LSTM.

6. Discusión y análisis

Responda con base en sus resultados:

- ¿Qué cambio de hiperparámetro tuvo el mayor impacto positivo en las métricas de validación de cada arquitectura? ¿Y el mayor impacto negativo?
- ¿La regularización (dropout, weight decay, gradient clipping) mejoró el desempeño en validación? ¿Qué método funcionó mejor para cada arquitectura y por qué creen que fue así?
- Comparando el MLP, la RNN y la LSTM, ¿cuál obtuvo mejor desempeño en test? ¿Cómo se relaciona esa diferencia con la capacidad de cada arquitectura para conservar el orden y las dependencias entre palabras?
- Con base en el experimento de la sección 4.1, ¿la RNN simple mostró señales de perder desempeño al aumentar la longitud de las secuencias en comparación con la LSTM? ¿Cómo relacionan esto con el problema de vanishing gradient discutido en clase?
- ¿En qué tipo de reseñas se equivocan más cada uno de los modelos (por ejemplo, reseñas muy largas, con sarcasmo, o con sentimiento mixto)? Analicen las matrices de confusión y algunos ejemplos mal clasificados.
- Con base en las ventajas y limitaciones discutidas en clase (complejidad, costo de entrenamiento, paralelización, cantidad de datos requerida, arquitecturas modernas como Transformers), si tuvieran que desplegar un modelo de producción para este problema priorizando tanto exactitud como eficiencia, ¿qué arquitectura elegirían y por qué?

Entregables

Entregable	Contenido
PDF (máx. 4 páginas)	Investigación de las capas de PyTorch para RNN/LSTM, tabla de resultados de las iteraciones (MLP, RNN y LSTM), comparación de arquitecturas, análisis de resultados y conclusiones.
Repositorio (Git)	Jupyter Notebook completo y comentado: carga y preparación de datos, investigación de capas, definición de los tres modelos (MLP, RNN y LSTM), entrenamiento de las 15+ iteraciones y evaluación final sobre test.

Incluyan el enlace al repositorio al final del PDF.